

CREATE TRIGGER (Transact-SQL)

Artículo • 01/02/2023 • Tiempo de lectura: 27 minutos

Se aplica a:  [SQL Server](#)  [Azure SQL Database](#)  [Azure SQL Managed Instance](#)

Crea un desencadenador DML, DDL o logon. Un desencadenador es un tipo especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en el servidor de bases de datos. Los desencadenadores DML se ejecutan cuando un usuario intenta modificar datos mediante un evento de lenguaje de manipulación de datos (DML). Los eventos DML son instrucciones INSERT, UPDATE o DELETE de una tabla o vista. Estos desencadenadores se activan cuando se desencadena cualquier evento válido, con independencia de que las filas de la tabla se vean o no afectadas. Para más información, consulte [DML Triggers](#).

Los desencadenadores DDL se ejecutan en respuesta a una variedad de eventos de lenguaje de definición de datos (DDL). Estos eventos corresponden principalmente a instrucciones CREATE, ALTER y DROP de Transact-SQL, y a determinados procedimientos almacenados del sistema que ejecutan operaciones de tipo DDL.

Los desencadenadores LOGON se activan en respuesta al evento LOGON que se genera cuando se establece la sesión de un usuario. Puede crear desencadenadores directamente a partir de instrucciones de Transact-SQL o de métodos de ensamblados creados en Common Language Runtime (CLR) de Microsoft .NET Framework y cargados en una instancia de SQL Server. SQL Server permite crear varios desencadenadores para cualquier instrucción específica.

Importante

El código malintencionado de los desencadenadores se puede ejecutar con privilegios concentrados. Para obtener más información sobre cómo mitigar este riesgo, vea [Administrar la seguridad de los desencadenadores](#).

Nota

En este artículo se describe la integración de CLR de .NET Framework en SQL Server. La integración de CLR no es válida con Azure SQL Database.

 [Convenciones de sintaxis de Transact-SQL](#)

Sintaxis de SQL Server

syntaxsql

```
-- SQL Server Syntax
-- Trigger on an INSERT, UPDATE, or DELETE statement to a table or view (DML
Trigger)
```

```
CREATE [ OR ALTER ] TRIGGER [ schema_name . ]trigger_name
ON { table | view }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
[ WITH APPEND ]
[ NOT FOR REPLICATION ]
AS { sql_statement [ ; ] [ ,...n ] | EXTERNAL NAME <method specifier [ ; ]
> }
```

```
<dml_trigger_option> ::=
    [ ENCRYPTION ]
    [ EXECUTE AS Clause ]
```

```
<method_specifier> ::=
    assembly_name.class_name.method_name
```

syntaxsql

```
-- SQL Server Syntax
-- Trigger on an INSERT, UPDATE, or DELETE statement to a
-- table (DML Trigger on memory-optimized tables)
```

```
CREATE [ OR ALTER ] TRIGGER [ schema_name . ]trigger_name
ON { table }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS { sql_statement [ ; ] [ ,...n ] }
```

```
<dml_trigger_option> ::=
    [ NATIVE_COMPILATION ]
    [ SCHEMABINDING ]
    [ EXECUTE AS Clause ]
```

syntaxsql

```
-- Trigger on a CREATE, ALTER, DROP, GRANT, DENY,
-- REVOKE or UPDATE statement (DDL Trigger)
```

```

CREATE [ OR ALTER ] TRIGGER trigger_name
ON { ALL SERVER | DATABASE }
[ WITH <ddl_trigger_option> [ ,...n ] ]
{ FOR | AFTER } { event_type | event_group } [ ,...n ]
AS { sql_statement [ ; ] [ ,...n ] | EXTERNAL NAME < method specifier > [
; ] }

<ddl_trigger_option> ::=
    [ ENCRYPTION ]
    [ EXECUTE AS Clause ]

```

syntaxsql

```

-- Trigger on a LOGON event (Logon Trigger)

CREATE [ OR ALTER ] TRIGGER trigger_name
ON ALL SERVER
[ WITH <logon_trigger_option> [ ,...n ] ]
{ FOR | AFTER } LOGON
AS { sql_statement [ ; ] [ ,...n ] | EXTERNAL NAME < method specifier > [
; ] }

<logon_trigger_option> ::=
    [ ENCRYPTION ]
    [ EXECUTE AS Clause ]

```

Sintaxis de Azure SQL Database

syntaxsql

```

-- Azure SQL Database Syntax
-- Trigger on an INSERT, UPDATE, or DELETE statement to a table or view (DML
Trigger)

CREATE [ OR ALTER ] TRIGGER [ schema_name . ]trigger_name
ON { table | view }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS { sql_statement [ ; ] [ ,...n ] [ ; ] > }

<dml_trigger_option> ::=
    [ EXECUTE AS Clause ]

```

syntaxsql

```
-- Azure SQL Database Syntax
-- Trigger on a CREATE, ALTER, DROP, GRANT, DENY,
-- REVOKE, or UPDATE STATISTICS statement (DDL Trigger)

CREATE [ OR ALTER ] TRIGGER trigger_name
ON { DATABASE }
    [ WITH <ddl_trigger_option> [ ,...n ] ]
{ FOR | AFTER } { event_type | event_group } [ ,...n ]
AS { sql_statement [ ; ] [ ,...n ] [ ; ] }

<ddl_trigger_option> ::=
    [ EXECUTE AS clause ]
```

❗ Nota

Para ver la sintaxis de Transact-SQL para SQL Server 2014 y versiones anteriores, consulte [Versiones anteriores de la documentación](#).

Argumentos

OR ALTER

Se aplica a: Azure SQL Database, SQL Server (a partir de SQL Server 2016 (13.x) SP1).

Altera el desencadenador condicionalmente solo si ya existe.

schema_name

El nombre del esquema al que pertenece un desencadenador DML. Los desencadenadores DML tienen como ámbito el esquema de la tabla o la vista donde se crean. *schema_name* no se puede especificar para los desencadenadores DDL o de inicio de sesión.

trigger_name

El nombre del desencadenador. El parámetro *trigger_name* debe cumplir las reglas de los [identificadores](#), con la excepción de que *trigger_name* no puede comenzar con los símbolos # o ##.

table | view

La tabla o la vista en que se ejecuta el desencadenador DML. A veces, se hace referencia a esta tabla o vista como tabla de desencadenador o vista de desencadenador, respectivamente. Especificar el nombre completo de la tabla o vista es opcional. Solo puede hacer referencia a una vista mediante un desencadenador INSTEAD OF. No puede definir desencadenadores DML en tablas temporales locales o globales.

DATABASE

Aplica el ámbito de un desencadenador DDL a la base de datos actual. Si se especifica, el desencadenador se activa cada vez que *event_type* o *event_group* tienen lugar en la base de datos actual.

ALL SERVER

Se aplica a: SQL Server 2008 (10.0.x) y versiones posteriores.

Aplica el ámbito de un desencadenador DDL o logon al servidor actual. Si se especifica, el desencadenador se activa cada vez que *event_type* o *event_group* tienen lugar en cualquier lugar del servidor actual.

WITH ENCRYPTION

Se aplica a: SQL Server 2008 (10.0.x) y versiones posteriores.

Ofusca el texto de la instrucción CREATE TRIGGER. El uso de WITH ENCRYPTION impide que el desencadenador se publique como parte de la replicación de SQL Server. WITH ENCRYPTION no se puede especificar para desencadenadores CLR.

EXECUTE AS

Especifica el contexto de seguridad en el que se ejecuta el desencadenador. Permite controlar qué cuenta de usuario utiliza la instancia de SQL Server para validar los permisos sobre cualquier objeto de base de datos al que haga referencia el desencadenador.

Esta opción es necesaria para los desencadenadores en tablas optimizadas para memoria.

Para obtener más información, vea [EXECUTE AS \(cláusula de Transact-SQL\)](#).

NATIVE_COMPILATION

Indica que el desencadenador está compilado de forma nativa.

Esta opción es necesaria para los desencadenadores en tablas optimizadas para memoria.

SCHEMABINDING

Garantiza que las tablas a las que un desencadenador hace referencia no se puedan quitar o modificar.

Esta opción es necesaria para desencadenadores en tablas optimizadas para memoria y no se admite en desencadenadores de tablas tradicionales.

FOR | AFTER

FOR o AFTER especifica que el desencadenador DML solo se activa cuando todas las

operaciones especificadas en la instrucción SQL desencadenadora se han iniciado correctamente. Además, todas las acciones referenciales en cascada y las comprobaciones de restricciones también deben ser correctas para que este desencadenador se active.

No puede definir desencadenadores AFTER en vistas.

INSTEAD OF

Especifica que se inicia el desencadenador DML *en lugar de* la instrucción SQL desencadenadora, por lo que se sobrescriben las acciones de las instrucciones desencadenadoras. No puede especificar INSTEAD OF para los desencadenadores DDL o LOGON.

Como máximo, puede definir un desencadenador INSTEAD OF por cada instrucción INSERT, UPDATE o DELETE en una tabla o vista. También puede definir otras vistas en las vistas que tengan su propio desencadenador INSTEAD OF.

No puede definir desencadenadores INSTEAD OF en vistas actualizables que usan WITH CHECK OPTION. Al hacerlo se genera un error cuando se agrega un desencadenador INSTEAD OF a una vista actualizable para la que se ha especificado WITH CHECK OPTION. Puede quitar esta opción mediante ALTER VIEW antes de definir el desencadenador INSTEAD OF.

{ [DELETE] [,] [INSERT] [,] [UPDATE] }

Especifica las instrucciones de modificación de datos que activan el desencadenador DML cuando se intenta en esta tabla o vista. Especifique al menos una opción. Use cualquier combinación de estas opciones en cualquier orden en la definición del desencadenador.

Para los desencadenadores INSTEAD OF, no puede utilizar la opción DELETE en tablas que tengan una relación referencial que especifica una acción ON DELETE en cascada. Tampoco se permite la opción UPDATE en tablas que tengan una relación referencial que especifica una acción ON UPDATE en cascada.

WITH APPEND

Se aplica a: de SQL Server 2008 (10.0.x) hasta SQL Server 2008 R2 (10.50.x).

Especifica que debe agregarse un desencadenador adicional de un tipo existente. WITH APPEND no se puede utilizar con desencadenadores INSTEAD OF o cuando se declara un desencadenador AFTER de forma explícita. Por motivos de compatibilidad con versiones anteriores, use solo WITH APPEND si se especifica FOR, sin INSTEAD OF o AFTER. No puede especificar WITH APPEND si usa EXTERNAL NAME (es decir, si se trata de un desencadenador CLR).

event_type

El nombre de un evento de lenguaje Transact-SQL que, después del inicio, hace que se active un desencadenador DDL. Los eventos válidos para los desencadenadores DDL se enumeran en [Eventos DDL](#).

event_group

El nombre de un agrupamiento predefinido de eventos de lenguaje TransactSQL. El desencadenador DDL se activa tras el inicio de cualquier evento de lenguaje Transact-SQL que pertenece a *event_group*. Los grupos de eventos válidos para los desencadenadores DDL se enumeran en [Grupos de eventos DDL](#).

Una vez que CREATE TRIGGER ha terminado de ejecutarse, *event_group* también actúa como una macro al agregar los tipos de eventos que comprende a la vista de catálogo sys.trigger_events.

NOT FOR REPLICATION

Se aplica a: SQL Server 2008 (10.0.x) y versiones posteriores.

Indica que el desencadenador no debe ejecutarse cuando un agente de replicación modifica la tabla involucrada en el mismo.

sql_statement

Las condiciones y acciones del desencadenador. Las condiciones del desencadenador especifican los criterios adicionales que determinan si los intentos de los eventos DML, DDL o LOGON dan lugar a la ejecución de las acciones del desencadenador.

Las acciones del desencadenador especificadas en las instrucciones Transact-SQL surten efecto cuando se intenta la operación.

Los desencadenadores pueden incluir cualquier número y tipo de instrucciones Transact-SQL, con excepciones. Para obtener más información, vea la sección Comentarios. Un desencadenador está diseñado para comprobar o cambiar los datos según una instrucción de modificación o definición de datos; no debe devolver datos al usuario. Las instrucciones Transact-SQL de un desencadenador incluyen a menudo [lenguaje de control de flujo](#).

Los desencadenadores DML usan las tablas lógicas (conceptuales) insertadas y eliminadas. Presentan una estructura similar a la tabla en que se define el desencadenador, es decir, la tabla en que se intenta la acción del usuario. Las tablas insertadas y eliminadas contienen los valores antiguos o nuevos de las filas que pueden modificarse mediante la acción del usuario. Por ejemplo, para recuperar todos los valores de la tabla `deleted`, utilice:

```
SELECT * FROM deleted;
```

Para obtener más información, vea [Usar las tablas insertadas y eliminadas](#).

Los desencadenadores DDL y logon capturan información acerca del evento desencadenador mediante el uso de la función [EVENTDATA \(Transact-SQL\)](#). Para obtener más información, vea [Usar la función EVENTDATA](#).

SQL Server permite actualizar las columnas **text**, **ntext** o **image** mediante el uso del desencadenador **INSTEAD OF** en tablas o vistas.

📌 Importante

Los tipos de datos **ntext**, **text** e **image** se quitarán en una versión futura de Microsoft SQL Server. Evite su uso en nuevos trabajos de desarrollo y piense en modificar las aplicaciones que los usan actualmente. Use **nvarchar(max)**, **varchar(max)** y **varbinary(max)** en su lugar. Tanto los desencadenadores **AFTER** como **INSTEAD OF** admiten los datos **varchar(MAX)**, **nvarchar(MAX)** y **varbinary(MAX)** en las tablas insertadas y eliminadas.

Para los desencadenadores en tablas optimizadas para memoria, el único *sql_statement* permitido en el nivel superior es un bloque **ATOMIC**. El código T-SQL permitido dentro del bloque **ATOMIC** está limitado por el código T-SQL permitido dentro de los procedimientos nativos.

< method_specifier > **Se aplica a:** SQL Server 2008 (10.0.x) y versiones posteriores.

En el caso de un desencadenador CLR, especifica el método de enlace de un ensamblado con el desencadenador. El método no debe tomar argumentos y debe devolver void. *class_name* debe ser un identificador válido de SQL Server y debe existir como clase en el ensamblado con visibilidad de ensamblado. Si la clase tiene un nombre calificado como espacio de nombres que utiliza '.' para separar las partes del espacio de nombres, el nombre de la clase debe estar delimitado por delimitadores de tipo [] o " ". La clase no puede ser anidada.

📌 Nota

De manera predeterminada, la capacidad de SQL Server de ejecutar código CLR está desactivada. Puede crear, modificar y quitar objetos de bases de datos que hagan referencia a módulos de código administrados, pero estas referencias no se

ejecutarán en una instancia de SQL Server a menos que la **opción `clr enabled`** esté habilitada mediante **`sp_configure`**.

Comentarios para los desencadenadores DML

Los desencadenadores DML se usan con frecuencia para aplicar las reglas de negocios y la integridad de datos. SQL Server proporciona integridad referencial declarativa (DRI) mediante las instrucciones `ALTER TABLE` y `CREATE TABLE`. Sin embargo, DRI no proporciona integridad referencial entre bases de datos. La integridad referencial se refiere a las reglas acerca de la relación entre la clave principal y la clave externa de las tablas. Para exigir la integridad referencial, utilice las restricciones de tipo `PRIMARY KEY` y `FOREIGN KEY` en `ALTER TABLE` y `CREATE TABLE`. Si existen restricciones en la tabla de desencadenadores, se comprueban después de la ejecución del desencadenador `INSTEAD OF` y antes de la de `AFTER`. Si se infringen las restricciones, se revierten las acciones del desencadenador `INSTEAD OF` y el desencadenador `AFTER` no se activa.

Puede especificar el primer y último desencadenador `AFTER` que se ejecuta en una tabla mediante `sp_settriggerorder`. Solo puede especificar el primer y último desencadenador `AFTER` para cada operación `INSERT`, `UPDATE` y `DELETE` de una tabla. Si hay otros desencadenadores `AFTER` en la misma tabla, se ejecutan aleatoriamente.

Si una instrucción `ALTER TRIGGER` modifica el primer o último desencadenador, se elimina el primer o último atributo establecido en el desencadenador modificado, y debe restablecer el valor del orden mediante `sp_settriggerorder`.

Un desencadenador `AFTER` se ejecuta solo después de ejecutar correctamente la instrucción SQL desencadenadora. La ejecución correcta incluye todas las acciones referenciales en cascada y las comprobaciones de restricciones asociadas al objeto actualizado o eliminado. Un desencadenador `AFTER` no activa de forma recursiva un desencadenador `INSTEAD OF` en la misma tabla.

Si un desencadenador `INSTEAD OF` definido en una tabla ejecuta una instrucción en la tabla que normalmente volvería a activarlo, al desencadenador no se le llama de forma recursiva. En su lugar, la instrucción se procesa como si la tabla no tuviera un desencadenador `INSTEAD OF` e inicia la cadena de operaciones de restricción y ejecuciones de desencadenadores `AFTER`. Por ejemplo, si un desencadenador se define como `INSTEAD OF INSERT` para una tabla. Además, el desencadenador ejecuta una instrucción `INSERT` en la misma tabla, y la instrucción `INSERT` iniciada por el desencadenador `INSTEAD OF` no vuelve a llamar al desencadenador. La instrucción `INSERT` iniciada por el desencadenador activa el proceso que ejecuta las acciones de restricción y activa cualquier desencadenador `AFTER INSERT` definido para la tabla.

Si un desencadenador **INSTEAD OF** definido en una vista ejecuta una instrucción en la vista que normalmente volvería a activarlo, no se llamará al desencadenador de forma recursiva. En su lugar, la instrucción se resuelve a modo de modificaciones en las tablas base subyacentes de la vista. En este caso, la definición de la vista debe cumplir todas las restricciones para una vista actualizable. Para obtener una definición de vistas actualizables, vea [Modificar datos mediante una vista](#).

Por ejemplo, si un desencadenador se define como **INSTEAD OF UPDATE** para una vista. Además, el desencadenador ejecuta una instrucción **UPDATE** que hace referencia a la misma vista, y la instrucción **UPDATE** iniciada por el desencadenador **INSTEAD OF** no vuelve a llamar al desencadenador. La instrucción **UPDATE** iniciada por el desencadenador se procesa en la vista, como si esta no tuviera un desencadenador **INSTEAD OF**. Las columnas que modifica la instrucción **UPDATE** deben resolverse en una única tabla base. Cada vez que se modifica una tabla base subyacente se inicia la cadena para aplicar restricciones y activar los desencadenadores **AFTER** definidos para la tabla.

Probar las acciones de **UPDATE** o **INSERT** en columnas específicas

Puede diseñar un desencadenador Transact-SQL que realice determinadas acciones según modificaciones de **UPDATE** o **INSERT** en columnas específicas. Para ello, utilice [UPDATE\(\)](#) o [COLUMNS_UPDATED](#) en el cuerpo del desencadenador. **UPDATE()** comprueba los intentos de **UPDATE** o **INSERT** en una columna. **COLUMNS_UPDATED** comprueba las acciones **UPDATE** o **INSERT** que se ejecutan en varias columnas. Esta función devuelve un patrón de bits que indica las columnas que se insertaron o se actualizaron.

Limitaciones de los desencadenadores

CREATE TRIGGER debe ser la primera instrucción en el proceso por lotes y solo se puede aplicar a una tabla.

Un desencadenador se crea solamente en la base de datos actual; sin embargo, un desencadenador puede hacer referencia a objetos que están fuera de la base de datos actual.

Si se especifica el nombre del esquema del desencadenador (para calificar el desencadenador), califique el nombre de la tabla de la misma forma.

La misma acción del desencadenador puede definirse para más de una acción del usuario (por ejemplo, **INSERT** y **UPDATE**) en la misma instrucción **CREATE TRIGGER**.

Los desencadenadores `INSTEAD OF DELETE/UPDATE` no pueden definirse en una tabla con una clave externa definida en cascada en la acción `DELETE/UPDATE`.

En un desencadenador se puede especificar cualquier instrucción `SET`. La opción `SET` seleccionada permanece en efecto durante la ejecución del desencadenador y, después, vuelve a su configuración anterior.

Cuando se activa un desencadenador, los resultados se devuelven a la aplicación que llama, exactamente igual que con los procedimientos almacenados. Para impedir que se devuelvan resultados a una aplicación debido a la activación de un desencadenador, no incluya las instrucciones `SELECT` que devuelven resultados ni las instrucciones que realizan una asignación de variable en un desencadenador. Un desencadenador que incluye tanto instrucciones `SELECT` que devuelven resultados al usuario como instrucciones que realizan la asignación de variables requiere un tratamiento especial. Debería escribir los resultados devueltos en todas las aplicaciones en que se permite modificar la tabla del desencadenador. Si es preciso que existan asignaciones de variable en un desencadenador, utilice una instrucción `SET NOCOUNT` al principio del mismo para impedir la devolución de cualquier conjunto de resultados.

Aunque una instrucción `TRUNCATE TABLE` es de hecho una instrucción `DELETE`, no activa un desencadenador porque la operación no registra eliminaciones de filas individuales. Sin embargo, solo los usuarios con permisos para ejecutar una instrucción `TRUNCATE TABLE` tienen que preocuparse de eludir inadvertidamente un desencadenador `DELETE` de esta manera.

La instrucción `WRITETEXT`, independientemente de que se registre o no, no activa un desencadenador.

Las siguientes instrucciones Transact-SQL no están permitidas en un desencadenador DML:

- `ALTER DATABASE`
- `CREATE DATABASE`
- `DROP DATABASE`
- `RESTORE DATABASE`
- `RESTORE LOG`
- `RECONFIGURE`

Además, las siguientes instrucciones Transact-SQL no se permiten en el cuerpo de un desencadenador DML cuando este se usa en la tabla o la vista que es objeto de la acción desencadenadora.

- `CREATE INDEX` (incluidos `CREATE SPATIAL INDEX` y `CREATE XML INDEX`)

- ALTER INDEX
- DROP INDEX
- DROP TABLE
- DBCC DBREINDEX
- ALTER PARTITION FUNCTION
- ALTER TABLE cuando se utiliza para hacer lo siguiente:
 - Agregar, modificar o quitar columnas.
 - Cambiar particiones.
 - Agregar o quitar restricciones de tipo PRIMARY KEY o UNIQUE.

ⓘ Nota

Ya que SQL Server no admite desencadenadores definidos por el usuario en tablas del sistema, se recomienda no crearlos.

Optimización de los desencadenadores DML

Los desencadenadores funcionan en transacciones (implícitas o no) y, mientras están abiertos, bloquean recursos. El bloqueo se mantiene hasta que la transacción se confirma (con COMMIT) o se rechaza (con ROLLBACK). Cuanto más tiempo se ejecuta un desencadenador, mayor es la probabilidad de que se bloquee otro proceso. Por lo tanto, escriba desencadenadores para reducir su duración siempre que sea posible. Una manera de conseguir acortar la duración consiste en liberar un desencadenador cuando una instrucción DML cambia cero filas.

Para liberar el desencadenador de un comando que no cambia ninguna fila, use la variable del sistema [ROWCOUNT_BIG](#).

El fragmento de código T-SQL siguiente muestra cómo liberar el desencadenador de un comando que no cambia ninguna fila. Este código debe estar presente al principio de cada desencadenador DML:

SQL

```
IF (ROWCOUNT_BIG() = 0)
RETURN;
```

Comentarios para los desencadenadores DDL

Los desencadenadores DDL, al igual que los estándar, inician procedimientos almacenados como respuesta a un evento. Pero, a diferencia de los desencadenadores estándar, no se ejecutan como respuesta a instrucciones UPDATE, INSERT o DELETE en una tabla o vista. En cambio, se ejecutan principalmente como respuesta a instrucciones de lenguaje de definición de datos (DDL). Los tipos de instrucciones incluyen CREATE, ALTER, DROP, GRANT, DENY, REVOKE y UPDATE STATISTICS. Algunos procedimientos almacenados del sistema que ejecutan operaciones de tipo DDL también pueden activar desencadenadores DDL.

📘 Importante

Pruebe los desencadenadores DDL para determinar sus respuestas a la ejecución de los procedimientos almacenados del sistema. Por ejemplo, la instrucción CREATE TYPE y los procedimientos almacenados sp_addtype y sp_rename activan un desencadenador DDL creado en un evento CREATE_TYPE.

Para obtener más información sobre los desencadenadores DDL, vea [Desencadenadores DDL](#).

Los desencadenadores DDL no se activan como respuesta a eventos que afectan a procedimientos almacenados y tablas temporales, ya sean locales o globales.

A diferencia de los desencadenadores DML, los desencadenadores DDL no tienen como ámbito los esquemas. Por tanto, no puede usar funciones como OBJECT_ID, OBJECT_NAME, OBJECTPROPERTY y OBJECTPROPERTYEX para efectuar consultas en metadatos sobre desencadenadores DDL. Utilice en su lugar las vistas de catálogo. Para más información, vea [Obtener información sobre los desencadenadores DDL](#).

📌 Nota

Los desencadenadores DDL con ámbito en el servidor aparecen en el Explorador de objetos de SQL Server Management Studio, en la carpeta **Desencadenadores**. Dicha carpeta se encuentra en la carpeta **Server Objects**. Los desencadenadores DDL de ámbito de base de datos aparecen en la carpeta **Desencadenadores de bases de datos**. Esta carpeta se encuentra en la carpeta **Programación** de la base de datos correspondiente.

Desencadenadores logon

Los desencadenadores LOGON realizan procedimientos almacenados en respuesta a un evento LOGON. Este evento se genera cuando se establece una sesión de usuario con una instancia de SQL Server. Los desencadenadores LOGON se activan después de que termine la fase de autenticación del inicio de sesión, pero antes de que se establezca la sesión de usuario. Por tanto, todos los mensajes que se originan dentro del desencadenador y que normalmente llegarían hasta el usuario, como los mensajes de error y los mensajes de la instrucción PRINT, se desvían al registro de errores de SQL Server. Para obtener más información, vea [Desencadenadores logon](#).

Los desencadenadores LOGON no se activan si se produce un error en la autenticación.

Los desencadenadores LOGON no admiten las transacciones distribuidas. Se devuelve el error 3969 cuando se activa un desencadenador LOGON que contiene una transacción distribuida.

Deshabilitar un desencadenador logon

Un desencadenador LOGON puede evitar la conexión a Motor de base de datos de todos los usuarios, incluidos los miembros del rol fijo de servidor **sysadmin**. Cuando el desencadenador LOGON evita que se realicen las conexiones, los miembros del rol fijo de servidor **sysadmin** pueden conectarse mediante la conexión de administrador dedicada o iniciando Motor de base de datos en modo de configuración mínima (-f). Para más información, consulte [Opciones de inicio del servicio de motor de base de datos](#).

Consideraciones generales sobre los desencadenadores

Devolver resultados

En una versión futura de SQL, se quitará la capacidad de devolver resultados desde los desencadenadores. Los desencadenadores que devuelven conjuntos de resultados pueden provocar un comportamiento inesperado en aplicaciones que no están diseñadas para utilizarlos. Evite la devolución de conjuntos de resultados desde desencadenadores en los nuevos trabajos de desarrollo y piense en modificar las aplicaciones que la usan actualmente. Para evitar que los desencadenadores devuelvan conjuntos de resultados, establezca la opción [No permitir resultados de desencadenadores](#) en 1.

Los desencadenadores LOGON siempre impiden que se devuelvan conjuntos de resultados y este comportamiento no se puede configurar. Si un desencadenador LOGON genera un conjunto de resultados, no se puede iniciar el desencadenador y se rechaza el intento de inicio de sesión activado por el desencadenador.

Desencadenadores múltiples

SQL Server permite crear varios desencadenadores para cada evento DML, DDL o LOGON. Por ejemplo, si se ejecuta `CREATE TRIGGER FOR UPDATE` para una tabla que ya tiene un desencadenador UPDATE, se crea un desencadenador de actualización adicional. En las versiones anteriores de SQL Server, solo se permitía un desencadenador por cada evento de modificación (INSERT, UPDATE, DELETE) en cada tabla.

Desencadenadores recursivos

SQL Server también admite la invocación recursiva de desencadenadores cuando el valor `RECURSIVE_TRIGGERS` está habilitado mediante `ALTER DATABASE`.

Los desencadenadores recursivos permiten dos tipos de repetición:

- Recursión indirecta

Con la repetición indirecta, una aplicación actualiza la tabla T1. Así se activa el desencadenador TR1 para actualizar la tabla T2. El desencadenador T2 activa y actualiza a continuación la tabla T1.

- Recursión directa

En la recursividad directa, una aplicación actualiza la tabla T1. Así se activa el desencadenador TR1 para actualizar la tabla T1. Debido a que la tabla T1 se ha actualizado, el desencadenador TR1 se activa de nuevo, y así sucesivamente.

En el ejemplo siguiente se usa la recursividad de desencadenadores directa e indirecta. Suponga que en la tabla T1 se han definido dos desencadenadores de actualización, TR1 y TR2. El desencadenador TR1 actualiza la tabla T1 recursivamente. Una instrucción UPDATE ejecuta cada TR1 y TR2 una vez. Además, el inicio de TR1 desencadena la ejecución de TR1 (recursivamente) y TR2. Las tablas `inserted` y `deleted` de un desencadenador específico contienen filas que corresponden solo a la instrucción UPDATE que invocó al desencadenador.



El comportamiento anterior solo se produce si el valor `RECURSIVE_TRIGGERS` está habilitado mediante `ALTER DATABASE`. No hay un orden establecido en el que se ejecuten los distintos desencadenadores definidos para un evento específico. Cada desencadenador debe ser independiente.

Deshabilitar `RECURSIVE_TRIGGERS` solo evita las recursiones directas. Para deshabilitar la repetición indirecta, establezca la opción de servidor de desencadenadores anidados en 0 con `sp_configure`.

Si alguno de los desencadenadores activa una instrucción `ROLLBACK TRANSACTION`, no se ejecuta ningún desencadenador posterior, independientemente del nivel de anidamiento.

Desencadenadores anidados

Puede anidar desencadenadores hasta un máximo de 32 niveles. Si un desencadenador cambia una tabla en la que hay otro desencadenador, el segundo se activa y puede, entonces, llamar a un tercero, y así sucesivamente. Si algún desencadenador de la cadena causa un bucle infinito, el nivel de anidamiento se habrá superado, con lo que se cancela el desencadenador. Cuando un desencadenador Transact-SQL inicia código administrado haciendo referencia a una rutina, un tipo o agregado CLR, esta referencia cuenta como un nivel en relación con el límite de anidamiento de 32 niveles. Los métodos invocados desde el código administrado no cuentan para este límite.

Para deshabilitar los desencadenadores anidados, establezca la opción `nested triggers` de `sp_configure` en 0 (desactivada). La configuración predeterminada admite desencadenadores anidados. Si los desencadenadores anidados están desactivados, los desencadenadores recursivos también se deshabilitan, independientemente del valor de `RECURSIVE_TRIGGERS` establecido mediante `ALTER DATABASE`.

El primer desencadenador `AFTER` anidado que esté dentro de un desencadenador `INSTEAD OF` se activará si la opción de configuración del servidor de **desencadenadores anidados** es 0. Sin embargo, con esta configuración, los desencadenadores `AFTER` posteriores no se activarán. Revise las aplicaciones en busca de desencadenadores anidados para determinar si las aplicaciones siguen las reglas de negocio cuando la configuración del servidor de **desencadenadores anidados** está establecida en 0. Si no es así, realice las modificaciones apropiadas.

Resolución diferida de nombres

SQL Server permite que los procedimientos almacenados, los desencadenadores y los procesos por lotes de Transact-SQL hagan referencia a tablas que no existen en tiempo de compilación. Esta capacidad se denomina resolución diferida de nombres.

Permisos

Para crear un desencadenador DML, es necesario contar con permiso ALTER en la tabla o vista en la que se crea el desencadenador.

Para crear un desencadenador DDL con ámbito de servidor (ON ALL SERVER) o un desencadenador LOGON, se requiere el permiso CONTROL SERVER en el servidor. Para crear un desencadenador DDL con ámbito en la base de datos (ON DATABASE), es necesario un permiso ALTER ANY DATABASE DDL TRIGGER en la base de datos actual.

Ejemplos

A. Usar desencadenador DML con un mensaje de aviso

El desencadenador DML siguiente imprime un mensaje en el cliente cuando alguien intenta agregar o cambiar datos en la tabla `Customer` de la base de datos AdventureWorks2019.

SQL

```
CREATE TRIGGER reminder1
ON Sales.Customer
AFTER INSERT, UPDATE
AS RAISERROR ('Notify Customer Relations', 16, 10);
GO
```

B. Usar un desencadenador DML con un mensaje de correo electrónico de aviso

Este ejemplo envía un mensaje de correo electrónico a una persona especificada (MaryM) cuando cambia la tabla `Customer`.

SQL

```
CREATE TRIGGER reminder2
ON Sales.Customer
AFTER INSERT, UPDATE, DELETE
AS
```

```
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'AdventureWorks2012 Administrator',
    @recipients = 'danw@Adventure-Works.com',
    @body = 'Don't forget to print a report for the sales force.',
    @subject = 'Reminder';

GO
```

C. Usar un desencadenador DML AFTER para exigir una regla de negocios entre las tablas PurchaseOrderHeader y Vendor

Debido a que las restricciones CHECK solo hacen referencia a las columnas en las que se han definido las restricciones de columna o de tabla, debe definir como desencadenadores cualquier restricción entre tablas, en este caso, las reglas de negocio.

En el ejemplo siguiente se crea un desencadenador DML en la base de datos AdventureWorks2012. Este desencadenador comprueba que la solvencia crediticia del proveedor es satisfactoria (no es 5) cuando se intenta insertar un nuevo pedido de compra en la tabla PurchaseOrderHeader. Para obtener la solvencia crediticia del proveedor, debe hacerse referencia a la tabla Vendor. Si la solvencia crediticia es demasiado baja, aparece un mensaje y no se ejecuta la inserción.

SQL

```
USE AdventureWorks2012;
GO
IF OBJECT_ID ('Purchasing.LowCredit','TR') IS NOT NULL
    DROP TRIGGER Purchasing.LowCredit;
GO
-- This trigger prevents a row from being inserted in the
Purchasing.PurchaseOrderHeader table
-- when the credit rating of the specified vendor is set to 5 (below ave-
rage).

CREATE TRIGGER Purchasing.LowCredit ON Purchasing.PurchaseOrderHeader
AFTER INSERT
AS
IF (ROWCOUNT_BIG() = 0)
RETURN;
IF EXISTS (SELECT 1
            FROM inserted AS i
            JOIN Purchasing.Vendor AS v
            ON v.BusinessEntityID = i.VendorID
            WHERE v.CreditRating = 5
        )
BEGIN
    RAISERROR ('A vendor's credit rating is too low to accept new
purchase orders.', 16, 1);
```

```
ROLLBACK TRANSACTION;
RETURN
END;
GO

-- This statement attempts to insert a row into the PurchaseOrderHeader ta-
ble
-- for a vendor that has a below average credit rating.
-- The AFTER INSERT trigger is fired and the INSERT transaction is rolled
back.

INSERT INTO Purchasing.PurchaseOrderHeader (RevisionNumber, Status, Emplo-
yeeID,
VendorID, ShipMethodID, OrderDate, ShipDate, SubTotal, TaxAmt, Freight)
VALUES (
2
,3
,261
,1652
,4
,GETDATE()
,GETDATE()
,44594.55
,3567.564
,1114.8638 );
GO
```

D. Usar un desencadenador DDL con ámbito de base de datos

En el ejemplo siguiente se usa un desencadenador DDL para impedir que se quiten sinónimos de una base de datos.

SQL

```
CREATE TRIGGER safety
ON DATABASE
FOR DROP_SYNONYM
AS
IF (@@ROWCOUNT = 0)
RETURN;
RAISERROR ('You must disable Trigger "safety" to remove synonyms!', 10,
1)
ROLLBACK
GO
DROP TRIGGER safety
ON DATABASE;
GO
```

E. Usar un desencadenador DDL con ámbito de servidor

En el ejemplo siguiente se utiliza un desencadenador DDL para imprimir un mensaje si se produce un evento CREATE DATABASE en la instancia actual del servidor, y se utiliza la función EVENTDATA para recuperar el texto de la instrucción Transact-SQL correspondiente. Para obtener más ejemplos que usan EVENTDATA en desencadenadores DDL, vea [Usar la función EVENTDATA](#).

Se aplica a: SQL Server 2008 (10.0.x) y versiones posteriores.

SQL

```
CREATE TRIGGER ddl_trig_database
ON ALL SERVER
FOR CREATE_DATABASE
AS
    PRINT 'Database Created.'
    SELECT EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand/CommandText)
[1]', 'nvarchar(max)')
GO
DROP TRIGGER ddl_trig_database
ON ALL SERVER;
GO
```

F. Usar un desencadenador LOGON

El ejemplo siguiente de desencadenador logon rechaza un intento de iniciar sesión en SQL Server como miembro del inicio de sesión *login_test* si ya hay tres sesiones de usuario ejecutándose con ese inicio de sesión.

Se aplica a: SQL Server 2008 (10.0.x) y versiones posteriores.

SQL

```
USE master;
GO
CREATE LOGIN login_test WITH PASSWORD = '3KHJ6dhx(0xVYsdf' MUST_CHANGE,
    CHECK_EXPIRATION = ON;
GO
GRANT VIEW SERVER STATE TO login_test;
GO
CREATE TRIGGER connection_limit_trigger
ON ALL SERVER WITH EXECUTE AS 'login_test'
FOR LOGON
AS
BEGIN
    IF ORIGINAL_LOGIN()= 'login_test' AND
        (SELECT COUNT(*) FROM sys.dm_exec_sessions
```

```
WHERE is_user_process = 1 AND  
      original_login_name = 'login_test') > 3  
  
ROLLBACK;  
END;
```

G. Ver los eventos que hacen que se active un desencadenador

En el ejemplo siguiente se efectúa una consulta en las vistas de catálogo `sys.triggers` y `sys.trigger_events` para determinar qué eventos de lenguaje Transact-SQL hacen que se active el desencadenador `safety`. El desencadenador, `safety`, se crea en el ejemplo "D", encontrado más arriba.

SQL

```
SELECT TE.*  
FROM sys.trigger_events AS TE  
JOIN sys.triggers AS T ON T.object_id = TE.object_id  
WHERE T.parent_class = 0 AND T.name = 'safety';  
GO
```

Consulte también

[ALTER TABLE \(Transact-SQL\)](#)
[ALTER TRIGGER \(Transact-SQL\)](#)
[COLUMNS_UPDATED \(Transact-SQL\)](#)
[CREATE TABLE \(Transact-SQL\)](#)
[DROP TRIGGER \(Transact-SQL\)](#)
[ENABLE TRIGGER \(Transact-SQL\)](#)
[DISABLE TRIGGER \(Transact-SQL\)](#)
[TRIGGER_NESTLEVEL \(Transact-SQL\)](#)
[EVENTDATA \(Transact-SQL\)](#)
[sys.dm_sql_referenced_entities \(Transact-SQL\)](#)
[sys.dm_sql_referencing_entities \(Transact-SQL\)](#)
[sys.sql_expression_dependencies \(Transact-SQL\)](#)
[sp_help \(Transact-SQL\)](#)
[sp_helptrigger \(Transact-SQL\)](#)
[sp_helptext \(Transact-SQL\)](#)
[sp_rename \(Transact-SQL\)](#)
[sp_settriggerorder \(Transact-SQL\)](#)

[UPDATE\(\) \(Transact-SQL\)](#)

[Obtener información acerca de los desencadenadores DML](#)

[Obtener información sobre los desencadenadores DDL](#)

[sys.triggers \(Transact-SQL\)](#)

[sys.trigger_events \(Transact-SQL\)](#)

[sys.sql_modules \(Transact-SQL\)](#)

[sys.assembly_modules \(Transact-SQL\)](#)

[sys.server_triggers \(Transact-SQL\)](#)

[sys.server_trigger_events \(Transact-SQL\)](#)

[sys.server_sql_modules \(Transact-SQL\)](#)

[sys.server_assembly_modules \(Transact-SQL\)](#)